=== UNIT 5: OBJECT-ORIENTED PROGRAMMING (OOP) IN PYTHON ===

>> 5.1 Object-Oriented Programming Paradigm Principles

Object-Oriented Programming (OOP) models software systems around real-world entities combining data state (attributes) and behavioral operations (methods):
* Four Cardinal Pillars of OOP: (1) Encapsulation: Bundling data and methods into a single class unit while restricting direct external access to internal state. (2) Abstraction: Exposing public interfaces while hiding complex implementation details. (3) Inheritance: Creating hierarchical classes that inherit and extend attributes and methods from base classes. (4) Polymorphism: Presenting a unified interface for entities of different underlying types.

>> 5.2 Classes, Instances and the Self Parameter

* Class Definition: A user-defined blueprint from which individual objects (instances) are instantiated: class ClassName: ...
* The __init__ Constructor Method: Special dunder (double-underscore) initialization method invoked automatically upon object creation to initialize instance attributes.
* The self Parameter: Explicit reference to the current instance invoking the method. Must be the first parameter of any instance method.
* Instance vs Class Attributes:
- Instance Attributes: Variables bound to self (self.x = 10) unique to each instance.
- Class (Static) Attributes: Variables declared directly within class body shared collectively across all instances.

>> 5.3 Encapsulation and Access Modifiers

Python enforces encapsulation through naming conventions rather than strict access keywords (public/private):
* Public Attributes: Standard identifiers accessible everywhere (self.name).
* Protected Attributes: Prefixed with a single underscore (_self._name). Indicates internal use; accessible by subclasses.
* Private Attributes & Name Mangling: Prefixed with double underscores (__self.__secret). CPython automatically mangles the name to _ClassName__secret to prevent accidental override by subclasses.
* Property Decorators (@property, @attr.setter): Implement pythonic getters and setters with validation logic without changing public access syntax.

>> 5.4 Inheritance and Method Resolution Order (MRO)

* Inheritance Types: Single, Multilevel, Hierarchical, and Multiple Inheritance.
* The super() Function: Dynamically delegates method calls to the parent or next sibling class in the inheritance chain: super().__init__(*args).
* Multiple Inheritance & Method Resolution Order (MRO): Determines method lookup sequence using the C3 Superclass Linearization Algorithm: ClassName.__mro__ or ClassName.mro(). Prevents the 'Diamond Problem'.

>> 5.5 Polymorphism and Magic (Dunder) Methods

* Operator Overloading via Dunder Methods: Custom classes integrate seamlessly with Python operators by overriding magic methods: __str__, __repr__, __len__, __add__, __sub__, __eq__, __getitem__.
* Duck Typing: 'If it walks like a duck and quacks like a duck, it's a duck.' Python checks for presence of required methods rather than explicit inheritance hierarchies.

[PROGRAM 1: Bank Account System with Encapsulation & Properties]

Problem Context: Implement a BankAccount class with private balance (__balance), validation for deposit/withdrawal operations, and @property decorators for balance access.

Python Source Code:

```
class BankAccount:
def __init__(self, account_holder, initial_balance=0.0):
self.holder = account_holder
self.__balance = max(0.0, float(initial_balance))

@property
def balance(self):
return self.__balance

def deposit(self, amount):
if amount <= 0:
raise ValueError('Deposit amount must be strictly positive.')
self.__balance += amount
return self.__balance
```

```
def withdraw(self, amount):
if amount <= 0:
raise ValueError('Withdrawal amount must be positive.')
if amount > self.__balance:
raise ValueError('Insufficient funds.')
self.__balance -= amount
return self.__balance

def __str__(self):
return f'Account[{self.holder}]: INR {self.__balance:,.2f}'

acc = BankAccount('Aarav Sharma', 25000)
acc.deposit(10000)
acc.withdraw(5000)
print(acc)
print(f'Current Balance (via property): INR {acc.balance:,.2f}')
```

Execution Trace & Output:
Initial: 25k -> Deposit 10k -> 35k -> Withdraw 5k -> 30k.
Output: 'Account[Aarav Sharma]: INR 30,000.00'. Direct access to acc.__balance raises AttributeError.

[PROGRAM 2: Inheritance Hierarchy for University Employees]

Problem Context: Create a base Person class, a derived Employee class, and specialized Faculty and Staff subclasses demonstrating super() and method overriding.
Python Source Code:

```
class Person:
def __init__(self, name, email):
self.name = name
self.email = email
def get_details(self):
return f'Name: {self.name} | Email: {self.email}'

class Faculty(Person):
def __init__(self, name, email, department, research_area):
super().__init__(name, email)
self.department = department
self.research_area = research_area
```

```
def get_details(self):
base = super().get_details()
return f'{base} | Dept: {self.department} | Research: {self.research_area}'

prof = Faculty('Dr. Ramanujan', 'raman@lpu.co.in', 'Computer Science', 'Quantum
Algorithms')
print(prof.get_details())
```

Execution Trace & Output:

Output: 'Name: Dr. Ramanujan | Email: raman@lpu.co.in | Dept: Computer Science |
Research: Quantum Algorithms'.

[PROGRAM 3: Operator Overloading with 2D Vector Dunder Methods]

Problem Context: Implement a 2D Vector class with dunder methods for addition (__add__), scalar multiplication (__mul__), string representation (__repr__), and magnitude equality (__eq__).

Python Source Code:

```
import math

class Vector2D:
def __init__(self, x, y):
self.x = float(x)
self.y = float(y)

def __add__(self, other):
return Vector2D(self.x + other.x, self.y + other.y)

def __mul__(self, scalar):
return Vector2D(self.x * scalar, self.y * scalar)

def magnitude(self):
return math.sqrt(self.x**2 + self.y**2)

def __eq__(self, other):
return math.isclose(self.magnitude(), other.magnitude())

def __repr__(self):
return f'Vector2D({self.x:.1f}, {self.y:.1f})'

v1 = Vector2D(3, 4)   # Magnitude = 5.0
```

```
v2 = Vector2D(1, 2)
v3 = v1 + v2          # Vector2D(4.0, 6.0)
v4 = Vector2D(0, 5)   # Magnitude = 5.0
print(f'v1 + v2 = {v3}')
print(f'v1 * 3  = {v1 * 3}')
print(f'|v1| == |v4|: {v1 == v4}')
```

Execution Trace & Output:
v1 + v2 = Vector2D(4.0, 6.0); v1 * 3 = Vector2D(9.0, 12.0); v1 == v4 evaluates to True.

[PROGRAM 4: Polymorphic Shape Area Processor]

Problem Context: Implement an abstract-like Shape base class and specialized Circle, Rectangle, and Triangle classes, processing areas polymorphically in a batch loop.
Python Source Code:

```
import math

class Shape:
def area(self): raise NotImplementedError('Subclass must implement area()')

class Circle(Shape):
def __init__(self, radius): self.r = radius
def area(self): return math.pi * self.r**2

class Rectangle(Shape):
def __init__(self, w, h): self.w = w; self.h = h
def area(self): return self.w * self.h

shapes = [Circle(5), Rectangle(4, 6), Circle(2.5)]
for s in shapes:
print(f'Shape {type(s).__name__:10s} | Area: {s.area():.2f}')
```

Execution Trace & Output:
Circle (r=5): 78.54; Rectangle (4x6): 24.00; Circle (r=2.5): 19.63.

>> 5.6 University Review & Examination Questions

* Q1: Explain Name Mangling in Python and describe how private attributes are accessed internally.
Answer: When an identifier inside a class begins with at least two leading underscores and at most one trailing underscore (e.g. __balance), CPython textually transforms

(mangles) the name by prefixing it with an underscore followed by the class name: _ClassName__attribute. This prevents accidental namespace collisions and unintended overriding by subclasses. It is NOT strict security: code can still access or mutate the mangled name via obj._ClassName__balance.

* Q2: Explain the C3 Linearization Algorithm and Method Resolution Order (MRO) in multiple inheritance.

Answer: When a class inherits from multiple parents, Python resolves method lookups along a deterministic sequence called the Method Resolution Order (MRO). Python uses the C3 Linearization Algorithm to compute MRO, ensuring three properties: (1) Children precede parents. (2) Multiple parent listing order is preserved. (3) Monotonicity: if class A precedes B in one class MRO, it must precede B in all derived MROs. The sequence can be inspected via Class.mro().

>> 5.7 Key Takeaways and Revision Points

* Classes encapsulate state and behavior; self explicitly passes the instance reference.
* Name mangling (__var -> _Class__var) protects private variables from subclass collision.
* super() enables clean cooperative multiple inheritance via MRO.
* Dunder methods (__add__, __len__, __str__) implement operator overloading and duck typing.